Boston University CS 630 Fall 2022
Review Problems

1 Network Flow

1. (a) Suppose you have a flow network $G = (V, E)$, such that every capacity c_e is an *even* number. Prove or disprove: there exists a maximum flow f on G that is *even* on every edge.

   (b) Suppose you have a flow network $G = (V, E)$, such that every capacity c_e is an *odd* number. Prove or disprove: There exists a maximum flow f on G that is *odd* on every edge.

2. Give a polynomial-time algorithm that, given a graph G with nonnegative integer edge capacities, determines if there is more than one minimum cut for G.

3. (Jeff Erickson page.363, exercise 5)

   (a) Describe a flow network that contains a *unique maximum (s, t)-flow* but does *not* contain a *unique minimum (s, t)-cut.*

   (b) Describe a flow network that contains a *unique minimum (s, t)-cut* but does *not* contain a *unique maximum (s, t)-flow.*

   (c) Describe an efficient algorithm to determine whether a given flow network contains a *unique minimum (s, t)-cut.*

   (d) Describe an efficient algorithm to determine whether a given flow network contains a *unique maximum (s, t)-flow. hint: this problem is related to whether or not there is a unique min-cut.*

4. What is the tightest bound you can give on the running time of the Ford-Fulkerson algorithm when the input graph satisfies the following: all vertices have degree at most D_{max}, and all edges have capacity at most C_{max} (where C_{max} is an integer)? Your bound should depend on n, C_{max} and D_{max} but nothing else.

5. Kleinberg-Tardos, Chapter 7, exercises 1–9 (page 415+).

6. Kleinberg-Tardos, Chapter 7, exercise 12 (page 420).

7. Kleinberg-Tardos, Chapter 7, exercise 15 (page 421).

8. Kleinberg-Tardos, Chapter 7, exercise 16 (page 422).

9. Kleinberg-Tardos, Chapter 7, exercise 19 (page 425).

10. Suppose you have already computed a maximum-value s-t-flow f for a graph $G = (V, E, c)$. Describe how, given $G, s, t, f, (u, v)$, you would update the flow in $O(m + n)$ time if the following changes were made to G:

    (a) The edge (u, v) with capacity 1 is added to the graph.

(b) The edge (u, v) with capacity 1 is deleted from the graph.

(c) The capacity of an existing edge (u, v) is *increased* by 1.

(d) The capacity of an existing edge (u, v) is *decreased* by 1.

11. During the pandemic, there is a global push to vaccinate as many people as quickly as possible. There are K different brands of vaccine available that need to be distributed to L countries. For each vaccine v_i there are q_i doses available. Further, we know the population p_j of each country c_j. However, not every vaccine is approved for use in every country. Currently, for each vaccine v_i there is a list ℓ_i of countries where it is approved.

    Find an algorithm to decide whether there is a way to immunize every person in every country with a vaccine approved by their country's health department.

    Your algorithm should use a reduction to network flow. Specify your algorithm by answering the questions below. Your answers should be clear enough that any other student could turn them in to working code. (Pictures are helpful.)

    (a) Design a directed graph $G(V, E)$ that is an instance of the max-flow problem. That is, G is directed, it has designated source s and sink t nodes and a capacity $c(e)$ on each directed edge. Describe each part of G:
    - Specify the vertices in G. *Hint: there should be nodes for both vaccines and countries.*
    - Describe the edges in G. Make clear which direction they are.
    - Describe the capacity on each edge.
    - Make sure there are vertices named s (source) and t (sink)

    (b) How many vertices does your graph have in terms of K and L (asymptotically)?

    (c) What is the maximum number of edges your graph can have, in terms of K and L (asymptotically)?

    (d) Suppose we found the maximum flow f in G. Describe how to find for each i, j the number of units of the vaccine v_i to be sent to country c_j given the value $f(e)$ along each edge.

    (e) Suppose that it is indeed possible to get each country its required amount of vaccine. In that case, what is the value of the flow that is returned by the max-flow subroutine?

12. During the pandemic, traveling nurses must be sent to places where infection rates are spiking. There are P different nurses practitioners, denoted by p_i, and there are H different hospitals that they can be assigned to. Obviously, each nurse can be assigned to at most one hospital. Each hospital h_j has a demand for d_j nurse practitioners. However, not every nurse is eligible to work at every hospital; for each nurse practitioner p_i we know the subset s_i of hospitals at which they are approved to work.

    Find an algorithm to decide whether there is a way to meet the demand for nurse practitioners at every hospital. Your algorithm should be a reduction to network flow. Specify your algorithm by answering each question below. Your answer should be clear enough that any other student could turn them in to working code.

(a) Design a directed graph $G(V, E)$ that is an instance of the max-Flow problem. That is, G is directed, it has designated source s and sink t nodes and a capacity $c(e)$ on each directed edge. Describe each part of G:

- Specify the vertices in G. *Hint: there should be nodes for both nurse practitioners and hospitals.*
- Describe the edges in G. Make clear which direction they are.
- Describe the capacity on each edge.

(b) How many vertices does your graph have in terms of P and H (asymptotically)?

(c) What is the maximum number of edges your graph can have, in terms of P and H (asymptotically)?

(d) Suppose we found the maximum flow f in G. Describe how to find for each nurse practitioner p_i which hospital they are sent to based on the flow values $f(e)$ along the edges.

(e) uppose that it is indeed possible to assign nurses so that all demands are met. In that case, what is the value of the flow that is returned by the max-flow subroutine?

2 Polynomial Time Reductions, P, NP, and NP-completeness

1. Prove that the following problems are in NP. You will do this by (0.) Formulate the decision version of each problem. (1.) Define the polynomial-length certificate and the polynomial-time verifier function that takes the problem and certificate as input. (2.) Prove that the certificate is a proof that the answer to the specific problem instance is "yes" . (3.) Prove that the function terminates on all "yes" instances of the problem and correctly verifies the certificate.

   (a) every polynomial problem we discussed this semester (e.g shortest paths with or without negative edge weights), variations on interval scheduling and partitioning, network flow and its applications, etc.)

   (b) Factoring

   (c) Longest Path

   (d) Hamiltonian-Path, Hamiltonian-Cycle, Traveling Sales Person (TSP)

   (e) Independent Set, Vertex Cover, Set Cover

   (f) Graph k-coloring (Given a graph G, can we assign at most k colors to its vertices, so that neighbors have different colors?)

   (g) 3-SAT

2. For each of the following statements, indicate whether it is true, false, or "unknown", where "unknown" indicates that scientists have not conclusively determined whether the statement is true or false. For example, the correct answer for the statement "$P = NP$" is "unknown". But the correct answer for "the best algorithm for the maximum flow problem in n-vertex graphs takes time at least 2^n" is "false".

   Give a short justification for your answer (i.e. explain why the statement is true, or false, or not known to be either true or false).

(a) If X is in NP, then there does not exist a polynomial-time algorithm for X.

(b) If X is NP-complete, then there is no polynomial-time algorithm for X.

(c) If X is NP-complete, Y is in NP, and X reduces to Y, then Y is NP-complete.

(d) MINIMUM-CUT is NP-complete.

(e) MINIMUM-CUT is in NP.

(f) MINIMUM-CUT is in P.

(g) Every problem in P is in NP.

(h) Every NP-complete problem is in NP.

(i) Some NP-complete problems have polynomial time algorithms, but some others do not.

3 Approximation Algorithms

1. Given an undirected graph $G(V, E)$ the *Maximum* Cut problem asks to partition the vertices into sets $A \subset V$ and $B \subset V$ such that the total number of edges connecting a node in A with a node in B is as large as possible. The notation $d(v, A)$ (or $d(v, B)$) denotes the number of edges running between node v and nodes in set A.

   Show that the following algorithm 1 is a 2-approximation algorithm to the Maximum Cut problem.

Algorithm 1: ApproxAlgo($G(V, E)$)

```
select nodes v_1, v_2 at random;
A ← {v_1}, B ← {v_2};
for v in V − {v_1, v_2} do
    if d(v, A) ≥ d(v, B) then
        A ← A ∪ {v};
    else
        B ← B ∪ {v};
return A,B
```

2. 2-Set Cover is a version of Set Cover where each item is contained in exactly two sets. Find a 2-approximation algorithm that finds a minimal set cover for this instance. *Hint: can you turn this into a graph problem?*

3. Consider the following algorithm for Bin Packing; items are considered in the fixed input order. The next item will be assigned to the most recent bin, if it doesn't fit then a new bin is started. Show that this is a 2-approximation algorithm. *Hint: look at the total sum of elements and the number of bins used.*

4. Consider the Restricted Length Load Balancing problem; we want to run n jobs, each of them has a length t_j. We have unlimited machines available, however all the jobs have to be finished within 24 hours. Find an assignment of jobs to machines so that all jobs are finished on time and as few machines are used as possible. Find a 2-approximation algorithm.